UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Engineering

SUBMITTED BY	**MIAN MUHAMMAD MOEEN**
ROLL NO	**2025(CYS-111-B)**
SECTION	**B**
WEEK NO	**3**
SUBMITTED TO	**SIR. HAFIZ MUHAMMAD MUBASHIR**
SEMESTER	**4TH**
SESSION	**2025-2029**
SUBJECT	**ARTIFICIAL INTELLIGENCE**

TASK NO. 1

REPORT:-

Part A: Spread of the infection

1. Python code

Copy this code into a Python file, for example `Home_Task_1.py`. It uses the network and device names from Appendix A.

CODE:-

```

from collections import deque

# Appendix A: Company network
network = {
    "N01": ["N02"],
    "N02": ["N01", "N03", "N04", "N30"],
    "N03": ["N02", "N04", "N05", "N06", "N07", "N12", "N13"],
    "N04": ["N02", "N03", "N08", "N09", "N10", "N11", "N14", "N15"],
    "N05": ["N03"],
    "N06": ["N03"],
    "N07": ["N03"],
    "N08": ["N04", "N09", "N18", "N23"],
    "N09": ["N04", "N08"],
    "N10": ["N04", "N11", "N17"],
    "N11": ["N04", "N10"],
    "N12": ["N03", "N16", "N17"],
    "N13": ["N03", "N18", "N19", "N20"],
    "N14": ["N04", "N21", "N22", "N23"],
    "N15": ["N04", "N24", "N25", "N26", "N27"],
    "N16": ["N12"],
    "N17": ["N10", "N12"],
    "N18": ["N08", "N13"],
    "N19": ["N13"],
    "N20": ["N13"],
    "N21": ["N14"],
    "N22": ["N14"],
    "N23": ["N08", "N14"],
    "N24": ["N15"],
```

```
    "N23": ["N08", "N14"],
    "N24": ["N15"],
    "N25": ["N15"],
    "N26": ["N15"],
    "N27": ["N15", "N28", "N29"],
    "N28": ["N27"],
    "N29": ["N27"],
    "N30": ["N02"]
}

device_names = {
    "N01": "Internet-Gateway",
    "N02": "Firewall-Main",
    "N03": "Core-Switch-1",
    "N04": "Core-Switch-2",
    "N05": "Web-Server",
    "N06": "Mail-Server",
    "N07": "DNS-Server",
    "N08": "Database-Server",
    "N09": "Auth-Server",
    "N10": "File-Server",
    "N11": "Backup-Server",
    "N12": "Switch-HR",
    "N13": "Switch-Finance",
    "N14": "Switch-Dev",
    "N15": "Switch-Lab",
    "N16": "HR-PC-1",
    "N17": "HR-PC-2",
    "N18": "Finance-PC-1",
    "N19": "Finance-PC-2",
    "N20": "Finance-Printer",
```

```
    "N19": "Finance-PC-2",
    "N20": "Finance-Printer",
    "N21": "Dev-PC-1",
    "N22": "Dev-PC-2",
    "N23": "Build-Server",
    "N24": "Lab-PC-1",
    "N25": "Lab-PC-2",
    "N26": "Lab-PC-3",
    "N27": "WiFi-AP",
    "N28": "Guest-Laptop-1",
    "N29": "Guest-Laptop-2",
    "N30": "IDS-Sensor"
}

def bfs(graph, start, ignored=None):
    visited = {start}
    queue = deque([start])
    parent = {start: None}
    distance = {start: 0}
    order = []

    while queue:
        node = queue.popleft()
        order.append(node)

        for neighbour in graph[node]:
            if neighbour == ignored:
                continue
            if neighbour not in visited:
                visited.add(neighbour)
                parent[neighbour] = node
```

```
                visited.add(neighbour)
                parent[neighbour] = node
                distance[neighbour] = distance[node] + 1
                queue.append(neighbour)

    return order, parent, distance

def build_path(parent, goal):
    if goal not in parent:
        return None

    path = []
    while goal is not None:
        path.append(goal)
        goal = parent[goal]

    return path[::-1]

def show_names(ids):
    return [device_names[n] for n in ids]

# Part A: Infection spread from N28
order, parent, distance = bfs(network, "N28")

print("1. BFS infection order:")
print(order)

print("\n2. Infection levels:")
for hour in range(max(distance.values()) + 1):
    devices = [n for n in order if distance[n] == hour]
    print("Hop", hour, ":", devices,
```

```
    devices = [n for n in order if distance[n] == hour]
    print("Hop", hour, ":", devices,
          "| Count:", len(devices))

print("\n3. Devices infected within each time limit:")
for hours in [2, 3, 4]:
    infected = [n for n in order if distance[n] <= hours]
    print(hours, "hours:", infected)
    print("Device names:", show_names(infected))

print("\n4. Database-Server infection time:",
      distance["N08"], "hours")

last_hour = max(distance.values())
last_devices = [
    n for n in order if distance[n] == last_hour
]
print("Farthest devices:", last_devices)
print("Names:", show_names(last_devices))
print("Last device in BFS order:", order[-1])

# Part B: Shortest paths
for goal in ["N08", "N16"]:
    path = build_path(parent, goal)
    print("\nBFS path to", device_names[goal])
    print(path)
    print(show_names(path))
    print("Hops:", len(path) - 1)

# Part C: BFS from the Internet-Gateway
gateway_order, _, gateway_distance = bfs(network, "N01")
```

```
145  gateway_order, _, gateway_distance = bfs(network, "N01")
146  farthest = max(gateway_distance.values())
147  print("\n7. Farthest from Internet-Gateway:",
148        farthest, "hops")
149  print([
150      n for n in gateway_order
151      if gateway_distance[n] == farthest
152  ])
153
154  # Part C: Remove one device at a time
155  for removed in ["N04", "N03", "N02"]:
156      reached, _, _ = bfs(network, "N01", ignored=removed)
157      unreachable = [
158          n for n in network if n not in reached
159      ]
160      print("\nRemoving", removed,
161            device_names[removed])
162      print("Unreachable IDs:", unreachable)
163      print("Unreachable names:", show_names(unreachable))
164      print("Number unreachable:", len(unreachable))
165
```

OUTPUT:-

```
1. BFS infection order:
['N28', 'N27', 'N15', 'N29', 'N04', 'N24', 'N25', 'N26', 'N02', 'N03', 'N08', 'N09', 'N10', 'N11', 'N14', 'N01', 'N30', 'N05', 'N06', 'N07', 'N12', 'N13', 'N18',

2. Infection levels:
Hop 0 : ['N28'] | Count: 1
Hop 1 : ['N27'] | Count: 1
Hop 2 : ['N15', 'N29'] | Count: 2
Hop 3 : ['N04', 'N24', 'N25', 'N26'] | Count: 4
Hop 4 : ['N02', 'N03', 'N08', 'N09', 'N10', 'N11', 'N14'] | Count: 7
Hop 5 : ['N01', 'N30', 'N05', 'N06', 'N07', 'N12', 'N13', 'N18', 'N23', 'N17', 'N21', 'N22'] | Count: 12
Hop 6 : ['N16', 'N19', 'N20'] | Count: 3

3. Devices infected within each time limit:
2 hours: ['N28', 'N27', 'N15', 'N29']
Device names: ['Guest-Laptop-1', 'WiFi-AP', 'Switch-Lab', 'Guest-Laptop-2']
3 hours: ['N28', 'N27', 'N15', 'N29', 'N04', 'N24', 'N25', 'N26']
Device names: ['Guest-Laptop-1', 'WiFi-AP', 'Switch-Lab', 'Guest-Laptop-2', 'Core-Switch-2', 'Lab-PC-1', 'Lab-PC-2', 'Lab-PC-3']
4 hours: ['N28', 'N27', 'N15', 'N29', 'N04', 'N24', 'N25', 'N26', 'N02', 'N03', 'N08', 'N09', 'N10', 'N11', 'N14']
Device names: ['Guest-Laptop-1', 'WiFi-AP', 'Switch-Lab', 'Guest-Laptop-2', 'Core-Switch-2', 'Lab-PC-1', 'Lab-PC-2', 'Lab-PC-3', 'Firewall-Main', 'Core-Switch-1',

4. Database-Server infection time: 4 hours
Farthest devices: ['N16', 'N19', 'N20']
Names: ['HR-PC-1', 'Finance-PC-2', 'Finance-Printer']
Last device in BFS order: N20
...
Removing N02 Firewall-Main
Unreachable IDs: ['N02', 'N03', 'N04', 'N05', 'N06', 'N07', 'N08', 'N09', 'N10', 'N11', 'N12', 'N13', 'N14', 'N15', 'N16', 'N17', 'N18', 'N19', 'N20', 'N21', 'N22
Unreachable names: ['Firewall-Main', 'Core-Switch-1', 'Core-Switch-2', 'Web-Server', 'Mail-Server', 'DNS-Server', 'Database-Server', 'Auth-Server', 'File-Server',
Number unreachable: 29
```

2. Full BFS infection order

BFS explores the closest devices first. Starting from infected device N28, the order is:

Hop-by-hop traversal order

N28 → N27 → N15 → N29 → N04 → N24 → N25 → N26 → N02 → N03 → N08 → N09 → N10 → N11 → N14 → N01 → N30 → N05 → N06 → N07 → N12 → N13 → N18 → N23 → N17 → N21 → N22 → N16 → N19 → N20

Total devices reached: 30

3. Infection levels

Each hop takes one hour, so hop 0 is the initially infected laptop.

Hop / hour	Device IDs reached	Count
0	N28	1
1	N27	1
2	N15, N29	2
3	N04, N24, N25, N26	4
4	N02, N03, N08, N09, N10, N11, N14	7
5	N01, N30, N05, N06, N07, N12, N13, N18, N23, N17, N21, N22	12
6	N16, N19, N20	3
Total	All devices	30

4. Devices infected within 2, 3 and 4 hours

- Within 2 hours — 4 devices: N28, N27, N15, N29.
- Within 3 hours — 8 devices: N28, N27, N15, N29, N04, N24, N25, N26.
- Within 4 hours — 15 devices: all devices infected at hops 0 through 4, namely N28, N27, N15, N29, N04, N24, N25, N26, N02, N03, N08, N09, N10, N11 and N14.

5. Database server and the last devices

- Database-Server (N08): infected after 4 hours, at hop 4.
- Farthest devices: N16 (HR-PC-1), N19 (Finance-PC-2) and N20 (Finance-Printer), reached after 6 hours.
- Last device in the exact BFS traversal order: N20, Finance-Printer.

Part B: Attack path and shortest route

5. Shortest paths using BFS

BFS finds the path with the fewest hops because every connection takes the same amount of time.

Destination	Path using device IDs	Path length
Database-Server (N08)	N28 → N27 → N15 → N04 → N08	4 hops
HR-PC-1 (N16)	N28 → N27 → N15 → N04 → N03 → N12 → N16	6 hops

The corresponding device names are:

- To Database-Server: Guest-Laptop-1 → WiFi-AP → Switch-Lab → Core-Switch-2 → Database-Server.
- To HR-PC-1: Guest-Laptop-1 → WiFi-AP → Switch-Lab → Core-Switch-2 → Core-Switch-1 → Switch-HR → HR-PC-1.

6. Compare DFS with BFS

Add this function to your Python file to run DFS using the left-to-right neighbour order specified by the lab's improved DFS example.

```python

def dfs(graph, start, goal):
    stack = [(start, None)]
    visited = set()
    parent = {}
    order = []

    while stack:
        node, par = stack.pop()

        if node in visited:
            continue

        visited.add(node)
        parent[node] = par
        order.append(node)

        if node == goal:
            break

        # Reverse the list so the first listed
        # neighbour is explored first.
        for neighbour in reversed(graph[node]):
            if neighbour not in visited:
                stack.append((neighbour, node))

    return order, parent

dfs_order, dfs_parent_map = dfs(network, "N28", "N08")
dfs_path = build_path(dfs_parent_map, "N08")

print("DFS path to Database-Server:")
print(dfs_path)
```

```python
    return order, parent

dfs_order, dfs_parent_map = dfs(network, "N28", "N08")
dfs_path = build_path(dfs_parent_map, "N08")

print("DFS path to Database-Server:")
print(dfs_path)
print(show_names(dfs_path))
print("DFS hops:", len(dfs_path) - 1)

```

The DFS path for this dataset is:

N28 → N27 → N15 → N04 → N02 → N03 → N13 → N18 → N08

Algorithm	Path length	Result
BFS	4 hops	Finds the shortest path
DFS	8 hops	Finds a longer path first

Explanation: BFS is better for finding the quickest attack route when every connection takes one hour. DFS follows a branch deeply before backtracking, so it does not guarantee the shortest path. In this example, DFS's first path to N08 takes 8 hops, while BFS finds a 4-hop path.

Part C: Defence planning

7. Farthest devices from Internet-Gateway (N01)

BFS from N01 gives these distances:

Hops from gateway	Devices
0	N01
1	N02
2	N03, N04, N30
3	N05, N06, N07, N08, N09, N10, N11, N12, N13, N14, N15
4	N16, N17, N18, N19, N20, N21, N22, N23, N24, N25, N26, N27
5	N28, N29

Answer: Guest-Laptop-1 (N28) and Guest-Laptop-2 (N29) are the farthest devices from the Internet-Gateway, at 5 hops.

8. Single point of failure

The following results come from running BFS from N01 while ignoring one device at a time.

Devices listed as unreachable include the removed device itself.

[1] Remove N04 — Core-Switch-2

8 devices become unreachable

N04, N15, N24, N25, N26, N27, N28, N29.

[2] Remove N03 — Core-Switch-1

4 devices become unreachable

N03, N05, N06, N07.

[3] Remove N02 — Firewall-Main

29 devices become unreachable

N02 and every device from N03 through N30. Only N01, the Internet-Gateway itself, remains reachable.

Which device is most critical?

Firewall-Main (N02) is the most critical device for connectivity from the Internet-Gateway in this dataset. It is the only listed connection between N01 and the rest of the network. Removing it isolates the gateway from the other 29 devices.

9. Which device should be isolated to slow the infection?

First choice: Guest-Laptop-1 (N28). It is the original infected device, so isolating it stops the malware from spreading to any other device.

If N28 cannot be isolated immediately, WiFi-AP (N27) is the next practical choice based on this graph. N27 is the only direct connection between N28 and the rest of the network. Disconnecting it prevents the infection from moving from N28 into the company network.

This recommendation is supported by the BFS result: without that connection, the infection cannot progress from N28 to N27, N15 or the other devices.

Final conclusion of report

BFS is the appropriate algorithm for this incident-response task because the malware takes one hour per hop and the team needs to know the infection's spread over time and the shortest routes to critical devices. The infection reaches the Database-Server in 4 hours and the farthest devices in 6 hours. Isolating the infected laptop N28 immediately is the strongest first response, while N02 is the most critical device for maintaining the gateway's connection to the rest of the network.

TASK NO. 3

1. Which data structure is used by BFS and which by DFS? What single line of the lab code decides this?

Answer:

- **BFS:** Uses a Queue (FIFO — First In, First Out).
- **DFS:** Uses a Stack (LIFO — Last In, First Out).

The line that decides the behavior is:

BFS

```
node = queue.pop(0)
```

DFS

```
node = stack.pop()
```

2. In the DFS code of Section 4.2 the output is A C G F B E D and not A B D E C F G. Explain exactly why, and how to change it?

Answer:

The DFS code adds B and C to the stack in that order. Since a stack follows LIFO, C is removed first. Then G is removed before F because G was added last.

To visit nodes from left to right, change the child-adding line to:

```
stack = stack + graph[node][::-1]
```

OUTPUT:-

A B D E C F G

Explanation: `[::-1]` reverses the children before adding them to the stack. This makes the leftmost child get processed first.

3. A graph has 1,000,000 nodes and the goal is known to be only 3 steps from the start. Which algorithm is the better choice? Which algorithm would use less memory on a very deep tree, and why?

Answer:

- Better algorithm: BFS.
- Less memory on a very deep tree: DFS, generally.

Explanation: BFS explores all nodes at distance 1, then distance 2, then distance 3. Therefore, it can find a nearby goal without exploring deep branches first.

DFS follows one branch deeply and may waste time exploring a long path. However, BFS can require substantial memory if the graph has many nodes at each level. DFS generally uses less search memory on a deep tree because it mainly stores the current path and unexplored siblings.

4. Give one situation where BFS can fail (run out of memory) and one situation where DFS can fail (never finish or give a very poor path)?

Answer:

BFS failure: A graph has millions of nodes at the same level. BFS stores many of them in its queue and may run out of memory.

DFS failure: A search follows an infinite branch or repeatedly explores a cycle without a visited set. It may never reach the goal. DFS can also find a much longer path than necessary.

Explanation: BFS's main weakness is memory consumption. DFS's main weaknesses are getting stuck in deep or infinite paths and not guaranteeing the shortest path.

5. Complete the table below for BFS and DFS?

Question	BFS	DFS
Data structure		
Finds the shortest path (unweighted)?		
Complete on a finite graph with a visited set?		
Time complexity		
Memory use		
One real-world use from this lab		

Answer:

Question	BFS	DFS
Data structure	Queue (FIFO)	Stack (LIFO) or recursion
Finds shortest path in an unweighted graph?	Yes	Not guaranteed
Complete on a finite graph with a visited set?	Yes	Yes
Time complexity on a graph	(O(V+E))	(O(V+E))
Memory use on a graph	(O(V))	(O(V)) with a visited set
One real-world use	Finding the fewest network hops	Scanning folders and subfolders

Explanation of the symbols:

- (V) = number of vertices (nodes).
- (E) = number of edges (connections).

Both algorithms can process every node and edge in \(O(V+E)\) time when implemented with appropriate graph structures and a visited set.

BFS guarantees a shortest path when all edges have equal cost. DFS does not.

6. Think of your own real-life problem (for example a university campus map, a family tree, a game, a website) and describe it as a graph: what are the nodes, what are the edges, and would you use BFS or DFS? Why?

Example: Finding the shortest route around a university campus

Problem: A student wants to travel from the university entrance to the computer science department using the fewest walking-path segments.

- **Nodes:** University entrance, library, cafeteria, science block and computer science department.
- **Edges:** Walking paths connecting these locations.
- **Start node**: University entrance.
- **Goal node:** Computer science department.
- **Algorithm**: BFS.

Why BFS?

- BFS explores locations level by level and finds the route with the fewest edges when each edge represents one equally weighted path segment.
- For example:

Entrance → Library → Computer Science Department

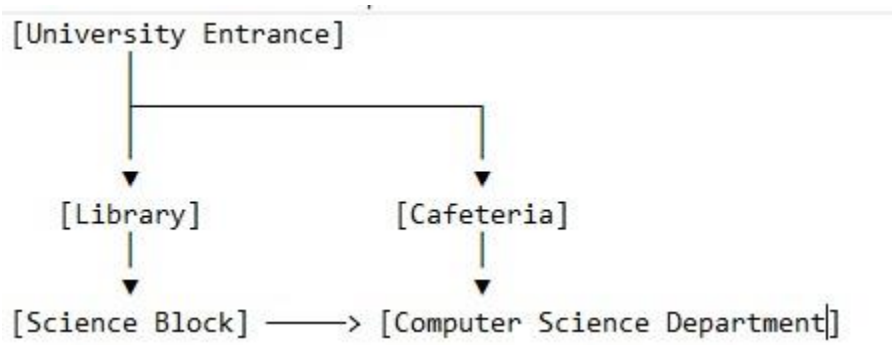
If the route has two edges and every alternative route has more edges, BFS finds the shortest route in terms of number of segments.

Important: If walking paths have different distances, ordinary BFS does not necessarily find the shortest distance in metres. A weighted shortest-path algorithm, such as Dijkstra's algorithm, would be more appropriate.

Graph:

Campus Graph Structure

- **Nodes (Locations):** University Entrance, Library, Cafeteria, Science Block, Computer Science Department.
- **Edges (Walking Paths):** Connections between the locations.



How BFS Explores This Graph Level-by-Level

- **Level 0 (Start):**
  - University Entrance
- **Level 1 (1 step away):**
  - Library, Cafeteria
- **Level 2 (2 steps away):**
  - Science Block, Computer Science Department

Because the goal (Computer Science Department) is reached at Level 2, BFS finds the valid path with the fewest segments:

University Entrance → Library → Computer Science Department